

EXPERIMENT 9: Word Embedding Visualization using PCA, t-SNE, and UMAP

AIM:

To generate random word embeddings for a sample corpus and visualize them using three dimensionality reduction techniques: Principal Component Analysis (PCA), t-Distributed Stochastic Neighbor Embedding (t-SNE), and UMAP.

PROCEDURE:

1. Import numpy, re, PCA from sklearn, TSNE from sklearn, matplotlib.
2. Define sample text and preprocess it.
3. Build vocabulary and create word2idx mapping.
4. Initialize random embeddings with dimension 5 for each word.
5. PCA: Apply PCA(n_components=2) on the embeddings.
6. t-SNE: Apply TSNE(n_components=2, perplexity=3) on embeddings.
7. UMAP (optional): Apply umap.UMAP(n_components=2) if available.
8. Define plot(data, title): scatter plot each word with its label.
9. Plot PCA and t-SNE results; try UMAP, print install message if unavailable.

CODE:

```
import numpy as np import re from
sklearn.decomposition import PCA from
sklearn.manifold import TSNE import
matplotlib.pyplot as plt
text = "language models learn patterns in text language processing helps computers
understand language"
def preprocess(text):
    text = text.lower()
    text = re.sub(r'^a-z\s|', '', text)
    return text.split()
tokens = preprocess(text)

vocab = list(set(tokens)) word2idx = {w:i
for i,w in enumerate(vocab)}
embedding_dim = 5
embeddings = np.random.rand(len(vocab), embedding_dim)
# PCA
pca = PCA(n_components=2)
pca_result = pca.fit_transform(embeddings)
```

```
# TSNE
tsne = TSNE(n_components=2, perplexity=3)
tsne_result = tsne.fit_transform(embeddings)

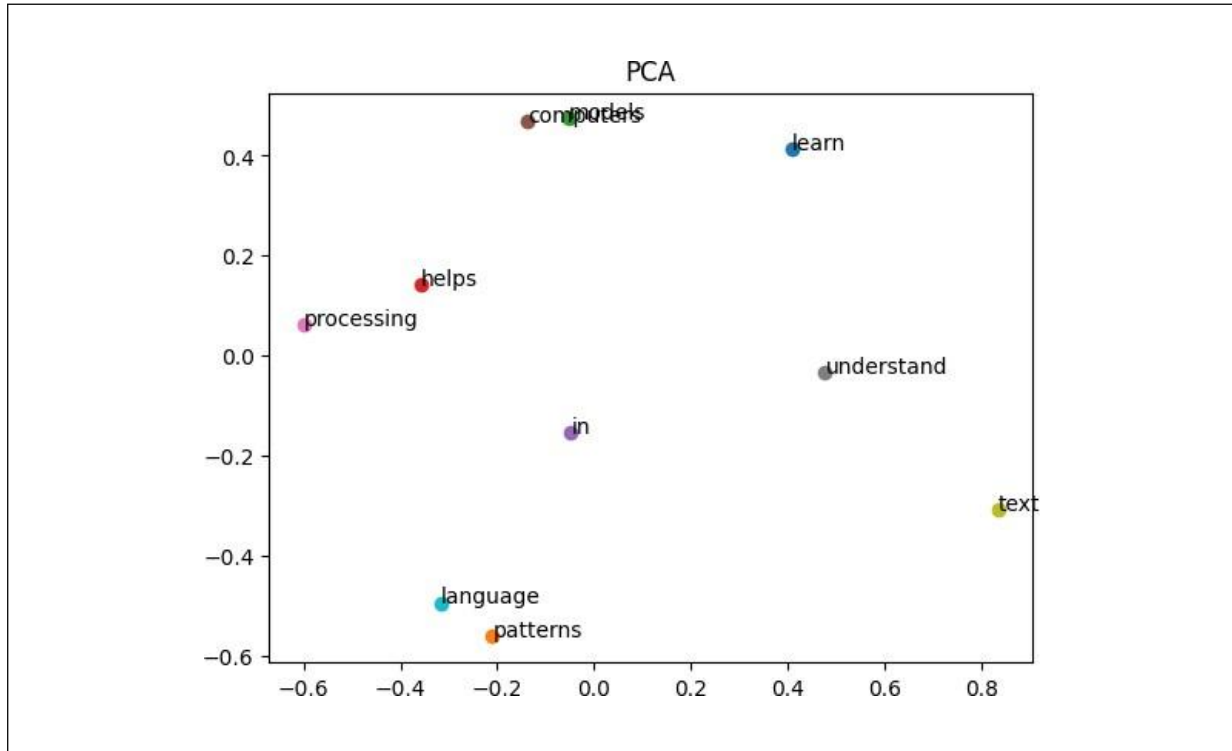
def plot(data, title):
    plt.figure()
    for i, word in enumerate(vocab):
        plt.scatter(data[i,0], data[i,1])
        plt.text(data[i,0], data[i,1], word)
    plt.title(title)
    plt.show()

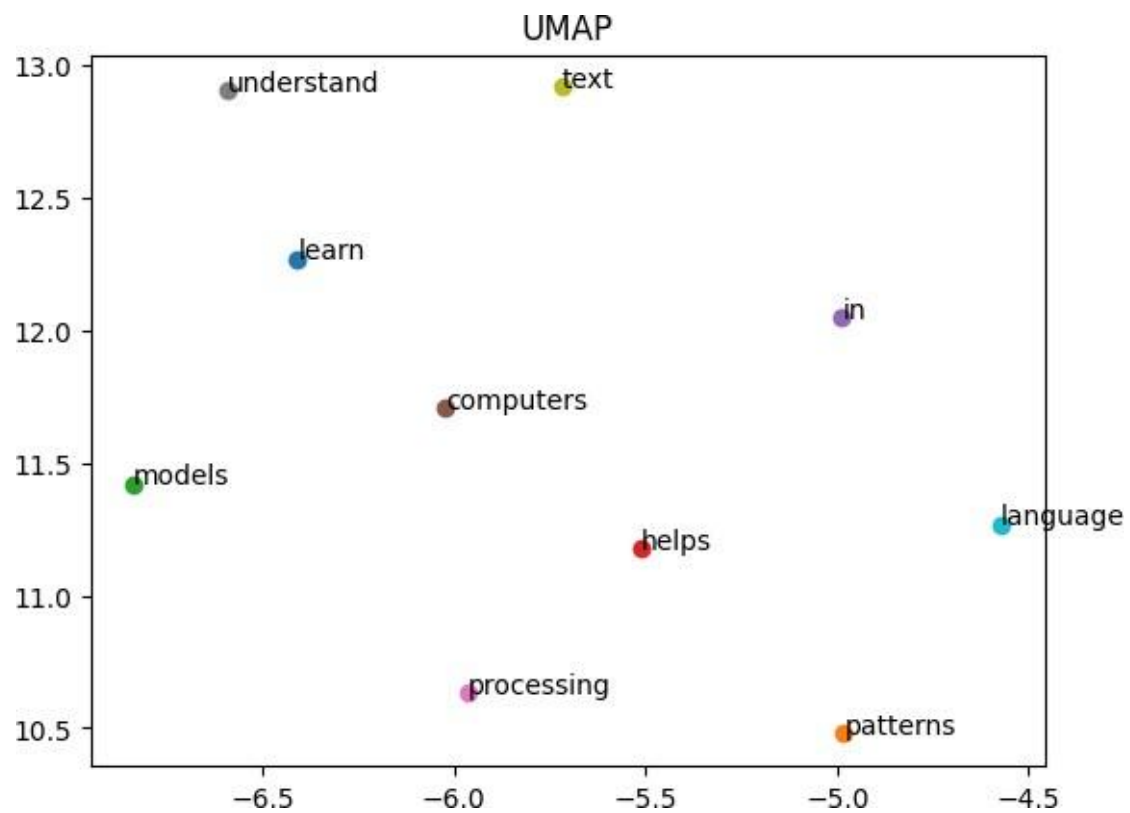
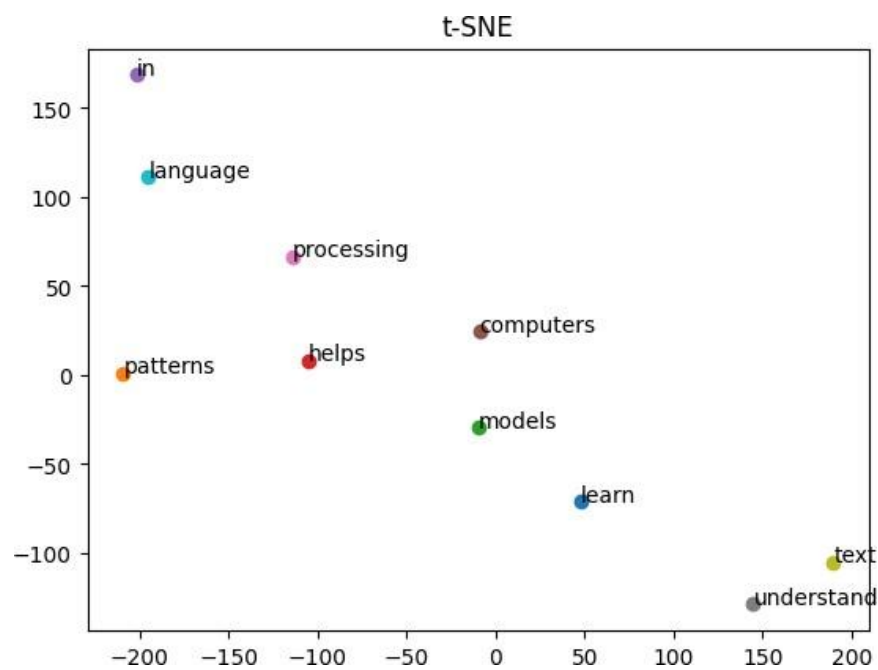
plot(pca_result, "PCA")
plot(tsne_result, "t-SNE")

try:
    import umap
    reducer = umap.UMAP(n_components=2)
    umap_result = reducer.fit_transform(embeddings)
    plot(umap_result, "UMAP")
except:
    print("Install UMAP: pip install umap-learn")
```

OUTPUT:

[PCA, t-SNE, and UMAP PLOTS - See Image Placeholder Below]





RESULT:

Word2Vec embeddings were successfully generated. Visualization techniques clearly showed word relationships.